

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 9

Docker Containerization

Docker · Multi-Stage Builds · Docker Compose · Health Checks · Production Images

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Part 1 — Introduction to Docker

Course: Spring Boot & Spring Cloud — Professional Edition

Learning Objectives

By the end of this part, you will be able to:

- Understand what Docker is.
- Explain why Docker was created.
- Identify the problems Docker solves.
- Differentiate between Docker and Virtual Machines (VMs).
- Understand the basic Docker architecture.
- Explain the Docker workflow:
 - Dockerfile
 - Docker Image
 - Docker Container
- Understand the role of Docker Hub.
- Recognize the main Docker components.

Before We Start...

Imagine you developed a Spring Boot application on your laptop.

Everything works perfectly.

You send it to another developer.

A few minutes later, they tell you:

"It doesn't work."

You ask:

"Why?"

They reply:

- I have Java 17 instead of Java 21.
- My Maven version is different.
- PostgreSQL is not installed.
- Redis is missing.
- Environment variables are different.
- My operating system is Linux while yours is Windows.

Sound familiar?

This is one of the biggest problems in software development.

This is exactly why Docker was created.

What is Docker?

Docker is a platform that packages an application together with everything it needs to run.

Instead of sending only your application, Docker packages:

- Your application
- Java Runtime
- Libraries
- Dependencies
- Configuration

Everything becomes a single package called a **Docker Image**.

The image can then run on any computer that has Docker installed.

Remember

Build once, Run anywhere.

Traditional Deployment

Before Docker, deploying applications was often difficult.

Example:

Spring Boot Application

↓

Install Java

↓

Install Maven

↓

Install PostgreSQL

↓

Install Redis

↓

Configure Environment Variables

↓

Run Application

Every machine needed to be configured manually.

Even a small difference could cause the application to fail.

Common Problems Without Docker

Without Docker, developers often face problems like:

- Different Java versions
- Missing libraries
- Missing database
- Different operating systems
- Incorrect environment variables
- Dependency conflicts

This is often called:

"It works on my machine."

How Docker Solves This Problem

Docker packages everything together.

Instead of sending only the application:

Application

+

Java Runtime

+

Libraries

+

Dependencies

+

Configuration

↓

Docker Image

The image behaves the same everywhere.

Whether you run it on:

- Windows
- Linux
- macOS
- Cloud Server

- AWS
- Azure
- Google Cloud

the application behaves exactly the same.

Why Do We Use Docker?

Docker provides many advantages.

1. Consistent Environment

Every developer uses the same environment.

No more:

"It works on my machine."

2. Fast Deployment

Instead of installing many tools manually,

simply run:

```
docker run ...
```

The application starts immediately.

3. Isolation

Each application runs inside its own container.

Applications do not interfere with each other.

For example:

Container 1

Java 17
PostgreSQL 15
Redis 7

Container 2

Java 21
PostgreSQL 16
MySQL

Both can run on the same machine without conflicts.

4. Lightweight

Docker containers are much smaller than Virtual Machines.

Typical Spring Boot image:

200 MB - 300 MB

Typical Virtual Machine:

Several GB

5. Fast Startup

Virtual Machine:

30-120 seconds

Docker Container:

1-5 seconds

6. Perfect for Microservices

A microservices application contains many services.

For example:

Gateway

↓

Config Server

↓

Eureka Server

↓

Product Service

↓

Order Service

↓

Payment Service

↓

Redis

↓

Kafka

↓

PostgreSQL

Docker allows every service to run independently.

Docker vs Virtual Machine

Many beginners think Docker is a Virtual Machine.

It is NOT.

Let's compare them.

Virtual Machine

A Virtual Machine contains a complete operating system.

Application

↓

Guest Operating System

↓

Hypervisor

↓

Host Operating System

↓

Hardware

Every VM has:

- Its own operating system
- Its own kernel
- Its own memory allocation

Advantages:

- Strong isolation
- Can run different operating systems

Disadvantages:

- Large size
- Slow startup
- High memory usage

Docker

Docker shares the Host Operating System kernel.

Application

↓

Docker Container

↓

Docker Engine

↓

Host Operating System

↓

Hardware

Containers only include what they actually need.

Advantages:

- Small
- Fast
- Lightweight
- Easy to deploy

Docker vs Virtual Machine Comparison

Virtual Machine	Docker
Includes full Operating System	Shares Host OS
Large size	Small size
Starts slowly	Starts in seconds
Uses more RAM	Uses less RAM
Heavy	Lightweight
Good for running different operating systems	Good for packaging applications

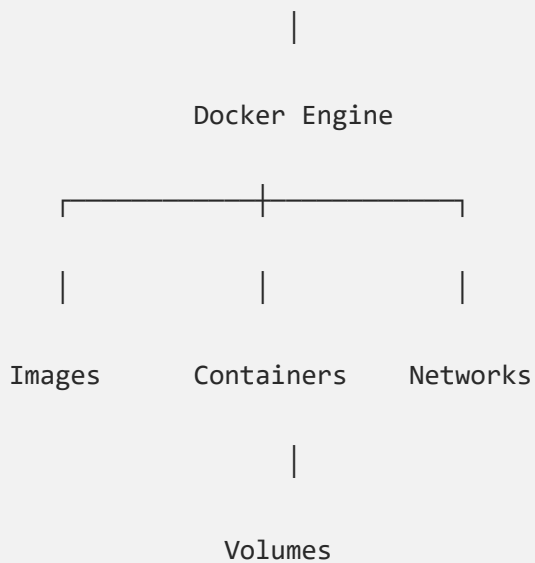
Docker Architecture

Docker consists of several components.

Developer

|

Docker CLI / Docker Desktop



Let's understand each component.

Docker CLI

CLI stands for:

Command Line Interface

This is where we type Docker commands.

Example:

```
docker images
```

```
docker ps
```

```
docker run nginx
```

Almost everything in Docker can be done using the CLI.

Docker Desktop

Docker Desktop is the graphical interface.

It allows you to:

- View Images
- View Containers
- Start Containers
- Stop Containers
- Delete Containers
- View Logs
- Manage Volumes
- Manage Networks

It is especially useful for beginners.

Docker Engine

Docker Engine is the heart of Docker.

It is responsible for:

- Building images
- Running containers
- Managing networks
- Managing volumes

Think of it as the Docker server running on your machine.

Docker Workflow

The Docker workflow is simple.

Step 1

Write Dockerfile

↓

Step 2

Build Image

↓

Step 3

Run Container

Let's understand each part.

What is a Dockerfile?

A Dockerfile is a text file.

It contains instructions that tell Docker how to build an image.

Example:

FROM Java Image

Copy Application

Run Application

You will learn how to write Dockerfiles later in this session.

What is a Docker Image?

A Docker Image is a package.

It contains:

- Application
- Java Runtime
- Libraries
- Dependencies
- Configuration

An image cannot execute by itself.

It is simply a template.

What is a Docker Container?

A Docker Container is a running instance of an image.

Think about it like this:

Docker Image

↓

`docker run`

↓

Running Container

The container is the actual running application.

Image vs Container

This is one of the most important concepts.

Docker Image

- Read-only
- Template
- Cannot execute by itself
- Can create many containers

Example:

Spring Boot Image

Docker Container

- Running application
- Can start
- Can stop
- Can restart
- Can be deleted

Example:

Product Service Container

A Simple Analogy

Think about object-oriented programming.

Class

↓

Create Object

↓

Object

Similarly:

Dockerfile

↓

Build

↓

Image

↓

Run

↓

Container

Another analogy:

Real Life	Docker
Cake Recipe	Dockerfile
Baked Cake	Image
Slice Being Eaten	Container

Docker Hub

Docker Hub is Docker's online image repository.

Think of it as GitHub, but for Docker Images.

It contains thousands of ready-made images.

Examples:

- Java
- PostgreSQL
- Redis
- Kafka
- RabbitMQ
- MongoDB
- NGINX
- MySQL

Instead of creating these images yourself, you simply download them.

Example

When you execute:

```
docker pull postgres
```

Docker downloads the PostgreSQL image from Docker Hub.

Similarly,

```
docker pull redis
```

downloads the Redis image.

Why Docker Hub is Useful

Without Docker Hub:

- Download PostgreSQL
- Install PostgreSQL
- Configure PostgreSQL

With Docker:

```
docker pull postgres
```

```
docker run postgres
```

Done!

Docker in Our Microservices Course

Throughout this course, we will use Docker to run:

- Config Server
- Eureka Server

- API Gateway
- Product Service
- Order Service
- Inventory Service
- Notification Service
- PostgreSQL
- Redis
- Kafka
- Zipkin

Eventually, we will start the entire platform with **one command** using Docker Compose.

Key Takeaways

- ✓ Docker packages applications with everything they need.
- ✓ Docker solves the "It works on my machine" problem.
- ✓ Docker Containers are lightweight.
- ✓ Docker is different from a Virtual Machine.
- ✓ Docker Images are templates.
- ✓ Docker Containers are running instances of images.
- ✓ Dockerfiles are used to build images.
- ✓ Docker Hub provides ready-made images.

Self-Check Questions

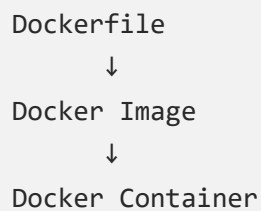
1. What problem does Docker solve?
2. Why is Docker popular for Microservices?
3. What is the difference between a Docker Image and a Docker Container?
4. Is Docker a Virtual Machine? Explain.
5. What is the purpose of a Dockerfile?
6. What is Docker Hub?

7. Can one Docker Image create multiple Containers?
8. Which is lighter: a Virtual Machine or a Docker Container?

Summary

In this part, you learned the fundamental concepts of Docker and why it has become the standard platform for deploying modern applications and microservices.

You now understand the Docker workflow:



```
graph TD; A[Dockerfile] --> B[Docker Image]; B --> C[Docker Container];
```

Dockerfile
↓
Docker Image
↓
Docker Container

In **Part 2**, we will install Docker, explore Docker Desktop, verify the installation, and run our first containers using real Docker commands. This will prepare us to containerize our Spring Boot microservices in the following parts.

Part 2 — Installing Docker & Running Your First Container

Course: Spring Boot & Spring Cloud — Professional Edition

Learning Objectives

By the end of this part, you will be able to:

- Install Docker on Windows, Linux, and macOS.
 - Verify that Docker is installed correctly.
 - Understand the difference between Docker Engine and Docker Desktop.
 - Open and use the Docker Command Line Interface (CLI).
 - Download your first Docker image.
 - Run your first Docker container.
 - Understand what happens when Docker executes a command.
-

Before Installing Docker

Docker can run on:

- Windows
- Linux
- macOS

Although the installation steps are slightly different, **the Docker commands are exactly the same on every operating system.**

This is one of Docker's biggest advantages.

Docker Engine vs Docker Desktop

Many beginners think these are the same thing.

They are related, but they are different.

Docker Engine

Docker Engine is the software that actually:

- Builds Docker images
- Runs Docker containers
- Creates networks
- Creates volumes

Think of Docker Engine as **the heart of Docker**.

Without Docker Engine, Docker cannot work.

Docker Desktop

Docker Desktop is a graphical application that includes Docker Engine and provides additional tools.

It offers:

- A graphical interface (GUI)
- Docker Engine
- Docker CLI
- Image management
- Container management
- Volume management
- Network management
- Log viewer

Docker Desktop is mainly used on **Windows** and **macOS**.

On most Linux distributions, Docker Engine is installed directly without Docker Desktop.

Installation Overview

Windows

↓
▼
Docker Desktop
↓
▼
Docker Engine + CLI

Linux
↓
▼
Docker Engine
↓
▼
Docker CLI

macOS
↓
▼
Docker Desktop
↓
▼
Docker Engine + CLI

Installing Docker on Windows

Step 1

Download **Docker Desktop** from the official Docker website.

(Your instructor will provide the official download link.)

Step 2

Run the installer.

Keep the default options.

Step 3

Enable **WSL2** if prompted.

What is WSL2?

WSL2 (Windows Subsystem for Linux) allows Docker to run Linux containers efficiently on Windows.

Step 4

Restart your computer if required.

Step 5

Open Docker Desktop.

Wait until Docker finishes starting.

When Docker is ready, you will see that the Docker Engine is running.

Installing Docker on Linux

Most Linux distributions already support Docker Engine.

Typical installation steps are:


```
sudo apt update
```

```
sudo apt install docker.io
```

Start Docker:

```
sudo systemctl start docker
```

Enable Docker to start automatically:

```
sudo systemctl enable docker
```

Optional

To avoid typing **sudo** every time:

```
sudo usermod -aG docker $USER
```

Log out and log in again.

Now Docker commands can be executed without sudo.

Installing Docker on macOS

Download Docker Desktop.

Install it like any normal macOS application.

Open Docker Desktop.

Wait until Docker Engine starts.

Verify the Installation

Open:

- Windows Terminal
- PowerShell
- Command Prompt
- Terminal (Linux/macOS)

Run:

```
docker --version
```

Example output:

```
Docker version 28.x.x
```

This confirms Docker is installed.

Check Docker Engine

Run:

```
docker info
```

Example output:

```
Server:
Containers: 0
Images: 0

Storage Driver:
Network:
```

This command displays information about the Docker Engine.

Check Docker is Running

If Docker Engine is not running, you may see an error like:

```
Cannot connect to the Docker daemon
```

This usually means Docker Desktop is not running or the Docker service has not started.

Simply start Docker and try again.

Your First Docker Command

Let's run our first Docker command.

```
docker version
```

Example:

```
Client:
  Version: ...

Server:
  Version: ...
```

Notice there are two parts:

- Client
- Server

The client sends commands.

The server (Docker Engine) executes them.

Docker CLI

CLI stands for:

Command Line Interface

Examples of Docker commands:

```
docker images
```

```
docker ps
```

```
docker run
```

```
docker build
```

Throughout this course, you will mainly use the CLI.

Running Your First Container

Let's run the simplest Docker container.

```
docker run hello-world
```

What Happens Internally?

When Docker executes this command:

```
docker run hello-world
```

Docker performs several steps automatically.

Step 1

Check if image exists



Step 2

Download image from Docker Hub

(if necessary)



Step 3
Create Container



Step 4
Start Container



Step 5
Display Output

All of this happens with a single command.

Expected Output

You should see a message similar to:

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

Congratulations!

You have just run your first Docker container.

Let's Try Something More Interesting

Instead of running a simple message, let's run a web server.

Execute:

```
docker run nginx
```

The terminal seems to "freeze."

Did Docker crash?

No.

The NGINX web server is running and waiting for requests.

Press:

```
CTRL + C
```

to stop it.

Running in Detached Mode

Normally, Docker attaches the terminal to the running application.

Instead, run:

```
docker run -d nginx
```

The **-d** option means:

Detached Mode

Docker runs the container in the background.

Now your terminal is free.

Viewing Running Containers

To see running containers:

```
docker ps
```

Example:

```
CONTAINER ID
```

```
IMAGE
```

```
STATUS
```

```
PORTS
```

```
NAMES
```

This shows all currently running containers.

Viewing All Containers

Some containers may have stopped.

To see everything:

```
docker ps -a
```

This displays:

- Running containers
- Stopped containers

Why Can't I Open NGINX?

Run:

```
docker run nginx
```

Then open:

```
http://localhost
```

Nothing appears.

Why?

Because Docker did **not** expose the web server to your computer.

The web server is running **inside the container**.

Understanding Port Mapping

Every application listens on a port.

NGINX listens on:

```
Port 80
```

But this port exists **inside** the container.

We must map it to our computer.

Port Mapping

Run:

```
docker run -p 8080:80 nginx
```

The format is:

Host Port : Container Port

Meaning:

8080 → Your Computer

80 → Inside Container

Open Your Browser

Now visit:

`http://localhost:8080`

You should see the NGINX welcome page.

Congratulations!

You have successfully exposed a container to your computer.

Understanding the Command

```
docker run -p 8080:80 nginx
```

Let's break it down.

Part	Meaning
------	---------

docker	Docker program
--------	----------------

run	Create and start a container
-----	------------------------------

-p	Publish a port
----	----------------

Part	Meaning
8080	Port on your computer
80	Port inside the container
nginx	Docker image to use

Running Multiple Containers

Docker allows multiple containers to run simultaneously.

Example:

Container 1

NGINX

Port 8080

Container 2

PostgreSQL

Port 5432

Container 3

Redis

Port 6379

Each container is isolated.

Stopping a Container

First, list running containers.

```
docker ps
```

Find the **Container ID**.

Example:

```
6f8d1c7ab34e
```

Stop it:

```
docker stop 6f8d1c7ab34e
```

You can also use the container name instead of the ID.

Starting a Stopped Container

```
docker start container_name
```

The container starts again without creating a new one.

Restarting a Container

```
docker restart container_name
```

This stops and starts the container in one command.

Removing a Container

If you no longer need a container:

```
docker rm container_name
```

Only stopped containers can be removed.

If the container is running, stop it first.

Common Beginner Mistakes

Mistake 1

Docker Desktop is not running.

Result:

```
Cannot connect to Docker daemon
```

Solution

Start Docker Desktop.

Mistake 2

Using a port that is already in use.

Example:

```
docker run -p 8080:80 nginx
```

If another application already uses port 8080, Docker will report an error.

Solution

Use another port.

Example:

```
docker run -p 9090:80 nginx
```

Mistake 3

Closing Docker Desktop.

Containers stop because Docker Engine is no longer running.

Hands-on Lab

Complete the following exercises.

Exercise 1

Verify Docker installation.

Run:

```
docker --version
```

```
docker info
```

Exercise 2

Run the Hello World container.

```
docker run hello-world
```

Exercise 3

Run an NGINX container.

```
docker run nginx
```

Stop it using **CTRL + C**.

Exercise 4

Run NGINX in detached mode.

```
docker run -d nginx
```

Display running containers.

```
docker ps
```

Exercise 5

Expose the web server.

```
docker run -p 8080:80 nginx
```

Open:

```
http://localhost:8080
```

Verify that the NGINX welcome page appears.

Key Takeaways

- ✓ Docker Desktop includes Docker Engine and useful graphical tools.
 - ✓ Docker Engine is responsible for building images and running containers.
 - ✓ `docker --version` verifies Docker is installed.
 - ✓ `docker info` provides information about the Docker Engine.
 - ✓ `docker run` creates and starts a container.
 - ✓ `-d` runs a container in the background.
 - ✓ `-p` maps a host port to a container port.
 - ✓ `docker ps` shows running containers.
 - ✓ `docker stop`, `docker start`, and `docker rm` manage the container lifecycle.
-

Self-Check Questions

1. What is the difference between Docker Engine and Docker Desktop?
 2. What does the `docker --version` command do?
 3. What happens internally when you execute `docker run hello-world`?
 4. Why can't you access an NGINX container from your browser without using `-p`?
 5. What does `-d` mean in the `docker run` command?
 6. What is the purpose of `docker ps`?
 7. Can you remove a running container? Why or why not?
 8. Explain the meaning of `8080:80` in the command `docker run -p 8080:80 nginx`.
-

Summary

In this part, you successfully installed Docker, verified that it was running correctly, and launched your first Docker containers. You also learned how to expose a container's ports so that services running inside the container can be accessed from your computer.

In **Part 3**, we will dive deeper into Docker Images and Containers, explore the complete container lifecycle, and learn the essential Docker commands that every Spring Boot developer uses daily.

Part 3 — Docker Images, Containers & Essential Docker Commands

Course: Spring Boot & Spring Cloud — Professional Edition

Learning Objectives

By the end of this part, you will be able to:

- Explain the difference between a Docker Image and a Docker Container.
- Understand the lifecycle of a Docker container.
- Download images from Docker Hub.
- Create, start, stop, restart, and delete containers.
- List images and containers.
- View container logs.
- Execute commands inside a running container.
- Remove unused Docker resources.
- Understand the most commonly used Docker commands.

Before We Begin

So far, we have used commands like:

```
docker run nginx
```

But what actually happens?

Where does **nginx** come from?

What is created?

What remains after the container stops?

To answer these questions, we first need to understand two fundamental Docker concepts:

- Docker Image
- Docker Container

These are the foundation of everything in Docker.

Docker Image

A **Docker Image** is a **read-only blueprint** used to create containers.

Think of it as a package that contains everything your application needs.

A Docker image includes:

- Application
- Runtime (Java, Node.js, Python...)
- Libraries
- Dependencies
- Configuration
- Startup instructions

An image **does not run** by itself.

It is simply a template.

Example

A Spring Boot image may contain:

Spring Boot Application

+

Java 21 Runtime

+

Required Libraries

+

Configuration

Docker stores all of this inside one image.

Docker Container

A **Docker Container** is a running instance of an image.

When Docker executes:

```
docker run nginx
```

Docker:

1. Uses the nginx image.
2. Creates a container.
3. Starts the container.

The container is the running application.

Image vs Container

Docker Image	Docker Container
Template	Running instance
Read-only	Read and write
Cannot execute	Running application
Can create many containers	Created from one image
Stored on disk	Running in memory

Simple Analogy

Imagine you own a bakery.

Recipe

↓

Bake Cake

↓

Eat Cake

Docker works similarly.

Dockerfile

↓

Build

↓

Image

↓

Run

↓

Container

Another analogy for Java developers:

Java Class

↓

new Object()

↓

Object

Docker:

Image

↓

`docker run`

↓

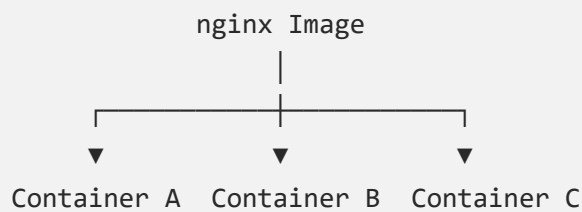
Container

One image can create many containers.

Can One Image Create Multiple Containers?

Absolutely.

Example:



Each container runs independently.

Where Do Images Come From?

Most images are downloaded from **Docker Hub**.

Examples:

- nginx
- postgres
- redis
- mysql
- rabbitmq
- kafka
- eclipse-temurin (Java)
- maven

Instead of creating everything yourself, you download ready-made images.

Downloading an Image

Use:

```
docker pull nginx
```

Docker downloads the image.

Download PostgreSQL:

```
docker pull postgres
```

Download Redis:

```
docker pull redis
```

Listing Downloaded Images

Run:

```
docker images
```

Example output:

REPOSITORY	TAG	IMAGE ID
nginx	latest	...
postgres	latest	...
redis	latest	...

Every downloaded image appears here.

Image Tags

Notice:

```
nginx:latest
```

The word after the colon is called the **tag**.

Example:

```
docker pull nginx:latest
```

or

```
docker pull postgres:16
```

Examples:

```
postgres:15
```

```
postgres:16
```

```
postgres:17
```

Tags usually represent:

- Version
- Release

- Environment

Best Practice

Avoid using `latest` in production.

Always specify the version you want.

Example:

```
docker pull postgres:16
```

Creating a Container

Containers are created using:

```
docker run
```

Example:

```
docker run nginx
```

Docker checks:

- Is the image available?
- If not, download it.
- Create a container.
- Start it.

Listing Running Containers

```
docker ps
```

Example:

```
CONTAINER ID
```

IMAGE

STATUS

PORTS

NAMES

Only running containers appear.

Listing All Containers

```
docker ps -a
```

This displays:

- Running containers
- Stopped containers
- Exited containers

Understanding Container Status

Containers can have different states.

Created

↓

Running

↓

Stopped

↓

Removed

A container moves through these states during its lifetime.

Container Lifecycle

`docker run`

↓

Running

↓

`docker stop`

↓

Stopped

↓

`docker start`

↓

Running

↓

`docker rm`

↓

Deleted

Remember:

Stopping a container does **not** delete it.

Starting a Stopped Container

```
docker start container_name
```

Example:

```
docker start wonderful_newton
```

Stopping a Container

```
docker stop container_name
```

or

```
docker stop container_id
```

Restarting a Container

```
docker restart container_name
```

Equivalent to:

Stop

↓

Start

Removing a Container

```
docker rm container_name
```

If Docker displays:

```
Container is running
```

Stop it first.

Removing Multiple Containers

```
docker rm container1 container2 container3
```

Viewing Container Logs

Containers continuously produce logs.

To view them:

```
docker logs container_name
```

Example:

```
docker logs nginx
```

Follow Live Logs

Just like Spring Boot console logs.

```
docker logs -f container_name
```

The **-f** means:

Follow

New log messages appear automatically.

Executing Commands Inside a Container

Sometimes you need to inspect a running container.

Use:

```
docker exec -it container_name bash
```

Example:

```
docker exec -it my-nginx bash
```

You are now inside the container.

Example:

```
ls  
  
pwd  
  
whoami
```

To exit:

```
exit
```

Note

Some lightweight images (such as Alpine Linux) do not include `bash`.

In that case, use:

```
docker exec -it container_name sh
```

Viewing Container Details

```
docker inspect container_name
```

This displays detailed information, including:

- IP Address
- Ports
- Volumes
- Environment Variables
- Network Configuration

Viewing Resource Usage

Monitor running containers.

```
docker stats
```

Example:

CPU

Memory

Network

Disk

Useful for monitoring applications.

Removing an Image

Images are removed using:

```
docker rmi image_name
```

Example:

```
docker rmi nginx
```

Docker will not remove an image if a container is still using it.

Remove the containers first.

Removing Unused Resources

Over time Docker stores many unused resources.

Clean everything using:

```
docker system prune
```

Docker removes:

- Stopped containers
- Unused networks
- Dangling images

Docker asks for confirmation before deleting.

Force Remove a Container

Sometimes a container refuses to stop.

Use:

```
docker rm -f container_name
```

Docker stops and removes it immediately.

Use this carefully.

Naming Containers

Docker automatically generates names.

Example:

```
adoring_einstein
```

```
happy_turing
```

```
kind_pasteur
```

You can choose your own name.

```
docker run --name product-service nginx
```

Now the container is called:

```
product-service
```

This is much easier to remember.

Common Beginner Mistakes

Mistake 1

Confusing images with containers.

Remember:

Image

↓

Creates

↓

Container

Mistake 2

Trying to remove an image before removing its containers.

Docker will display an error.

Always remove containers first.

Mistake 3

Using random container names.

Use meaningful names.

Example:

```
gateway
```

```
config-server
```

```
eureka
```

```
postgres
```

```
redis
```

This becomes very important in microservices.

Mistake 4

Forgetting to stop unused containers.

Unused containers consume resources.

Regularly check:

```
docker ps -a
```

Docker Commands Cheat Sheet

Command	Description
<code>docker pull nginx</code>	Download an image
<code>docker images</code>	List images
<code>docker run nginx</code>	Create and start a container
<code>docker ps</code>	Running containers
<code>docker ps -a</code>	All containers
<code>docker stop container</code>	Stop container
<code>docker start container</code>	Start container
<code>docker restart container</code>	Restart container
<code>docker logs container</code>	View logs
<code>docker logs -f container</code>	Follow logs
<code>docker exec -it container bash</code>	Enter a container
<code>docker inspect container</code>	Show container details

Command	Description
<code>docker stats</code>	Show resource usage
<code>docker rm container</code>	Remove container
<code>docker rmi image</code>	Remove image
<code>docker system prune</code>	Remove unused resources

Hands-on Lab

Exercise 1

Download three images.

```
docker pull nginx  
  
docker pull postgres:16  
  
docker pull redis:7
```

Verify:

```
docker images
```

Exercise 2

Run an NGINX container.

```
docker run -d --name nginx-demo -p 8080:80 nginx
```

Verify:

```
docker ps
```

Exercise 3

View the logs.

```
docker logs nginx-demo
```

Exercise 4

Open a shell inside the container.

```
docker exec -it nginx-demo bash
```

Run:

```
pwd
```

```
ls
```

Exit:

```
exit
```

Exercise 5

Stop and restart the container.

```
docker stop nginx-demo
```

```
docker start nginx-demo
```

Exercise 6

Remove the container.

```
docker stop nginx-demo
```

```
docker rm nginx-demo
```

Verify:

```
docker ps -a
```

Key Takeaways

- ✓ An **Image** is a template.
- ✓ A **Container** is a running instance of an image.
- ✓ One image can create many containers.
- ✓ Images are downloaded from Docker Hub.
- ✓ Containers have a lifecycle: Create → Run → Stop → Start → Remove.
- ✓ Use meaningful names for containers.
- ✓ `docker logs` and `docker exec` are two of the most useful commands for debugging applications.

Self-Check Questions

1. What is the difference between a Docker Image and a Docker Container?
2. Can one image create multiple containers?

3. What command downloads an image from Docker Hub?
4. What is the difference between `docker ps` and `docker ps -a`?
5. How do you view the logs of a running container?
6. How do you open a shell inside a container?
7. Why can't you remove an image that is still used by a container?
8. Why is it a good practice to give containers meaningful names?

Summary

You now understand the two most important Docker concepts—**Images** and **Containers**—and you have learned the essential Docker commands that you'll use throughout the rest of this course. In the next part, you'll build your own Docker images by writing a **Dockerfile**, learn about image layers, build context, `.dockerignore`, and prepare your Spring Boot application for containerization.

Part 4 — Building Docker Images with Dockerfile

Course: Spring Boot & Spring Cloud — Professional Edition

Learning Objectives

By the end of this part, you will be able to:

- Understand what a Dockerfile is.
- Write a simple Dockerfile.
- Understand the most common Dockerfile instructions.
- Build your own Docker image.
- Understand Build Context.
- Use a `.dockerignore` file.
- Understand Docker Image Layers.
- Learn how Docker caching speeds up image builds.
- Build a Docker image for a Spring Boot application.

Introduction

Until now, we have been using ready-made images from Docker Hub.

For example:

```
docker pull nginx
```

```
docker pull postgres
```

```
docker pull redis
```

But what about **our own Spring Boot application**?

How do we create an image for it?

The answer is:

Dockerfile

What is a Dockerfile?

A **Dockerfile** is a text file that contains instructions telling Docker how to build an image.

Think of it as a recipe.

Just like a cooking recipe tells a chef how to prepare a meal, a Dockerfile tells Docker how to prepare an image.

Simple Workflow

Write Dockerfile



`docker build`



Docker Image



`docker run`



Docker Container

Our First Dockerfile

Create a file named:

Dockerfile

Important

The file name is exactly **Dockerfile**.

- No extension (`.txt`, `.java`, etc.).
- Capital **D** is the common convention.

Add the following content:

```
FROM eclipse-temurin:21-jre

COPY app.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Let's understand each line.

FROM

```
FROM eclipse-temurin:21-jre
```

This line tells Docker:

Start from an existing image.

Here we use a Java Runtime Environment (JRE) image.

This image already contains Java 21.

Without this line, Docker wouldn't know which operating system or runtime to use.

COPY

```
COPY app.jar app.jar
```

This copies:


```
app.jar
```

from your computer into the Docker image.

After the build, the image contains your Spring Boot application.

ENTRYPOINT

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

This command runs automatically when the container starts.

Equivalent to:

```
java -jar app.jar
```

Visualizing the Build Process

```
graph TD
    A[Dockerfile] --> B[FROM Java Image]
    B --> C[COPY app.jar]
    C --> D[ENTRYPOINT]
    D --> E[Docker Image]
```

Building the Image

Assume your project contains:

```
product-service/
```

```
|— Dockerfile
```

```
|— app.jar
```

Open a terminal inside the project folder.

Run:

```
docker build -t product-service .
```

Breaking Down the Command

```
docker build -t product-service .
```

Part	Meaning
docker	Docker program
build	Build an image
-t	Assign a tag (name)
product-service	Image name
.	Current folder (Build Context)

What is Build Context?

This is one of the most misunderstood Docker concepts.

Look at this command:

```
docker build .
```

What does the **dot** (.) mean?

It means:

Send the **current folder** to Docker.

Everything inside the folder becomes available during the build.

Suppose your project looks like this:

```
product-service/
```

```
|— Dockerfile  
|— pom.xml  
|— src/  
|— target/  
|— .idea/  
|— README.md
```

Running:

```
docker build .
```

sends **everything** to Docker.

Why is This a Problem?

Some folders are unnecessary.

Examples:

```
target/
```

```
.idea/
```

```
.git/
```

```
logs/
```

These folders increase:

- Build time
- Image size

This is why we use:

```
.dockerignore
```

What is .dockerignore?

`.dockerignore` works like `.gitignore`.

It tells Docker which files should **not** be sent during the build.

Example:

```
target/  
  
.idea/  
  
.git/  
  
*.log
```

Now Docker ignores these files.

Why Use .dockerignore?

Benefits:

- Faster builds
- Smaller build context
- Smaller images
- Better security

- Less network traffic

Best Practice

Always create a `.dockerignore` file for every project.

Viewing Your Image

After building:

```
docker images
```

Example:

REPOSITORY	TAG
product-service	latest
nginx	latest
postgres	16

Your image is now available.

Running Your Image

Suppose your image is named:

```
product-service
```

Run:

```
docker run product-service
```

Docker creates a container from your image.

Giving the Container a Name

Instead of a random name:

```
docker run --name product-service product-service
```

Now the container is easier to identify.

Exposing Application Ports

Most Spring Boot applications use:

```
8080
```

Run:

```
docker run -p 8080:8080 product-service
```

Meaning:

```
Host Port      8080
```

↓

```
Container Port 8080
```

Now open:

```
http://localhost:8080
```

Your Spring Boot application should be accessible.

Docker Image Layers

One of Docker's biggest advantages is its **layered architecture**.

Each Dockerfile instruction creates a new layer.

Example:

```
FROM eclipse-temurin:21-jre

COPY app.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Creates:

```
Layer 1
Java Runtime

↓

Layer 2
Application

↓

Layer 3
Startup Command
```

Docker stores each layer separately.

Why Layers Are Important

Suppose you change only:

```
app.jar
```

Docker does **not** rebuild everything.

It rebuilds only the affected layer.

This makes builds much faster.

Docker Build Cache

Docker automatically caches layers.

Example:

```
FROM eclipse-temurin:21-jre

COPY app.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

First build:

Layer 1 → Built

Layer 2 → Built

Layer 3 → Built

Second build (only `app.jar` changed):

Layer 1 → Cached ⓘ

Layer 2 → Rebuilt

Layer 3 → Cached (if unchanged)

This significantly reduces build time.

Why Layer Order Matters

Consider this Dockerfile:


```
FROM eclipse-temurin:21-jre
```

```
COPY . .
```

```
RUN mvn package
```

Every time **any file changes**, Docker copies the entire project again and reruns Maven.

This is slow.

A better approach is:

```
COPY pom.xml .
```

```
RUN mvn dependency:go-offline
```

```
COPY src ./src
```

Now:

- If only Java files change, dependencies are reused.
- Maven dependencies are downloaded only when `pom.xml` changes.

This is much faster.

Note

We will use this optimization later with **multi-stage builds**.

Common Dockerfile Instructions

Instruction	Purpose
FROM	Base image
COPY	Copy files into the image
WORKDIR	Set the working directory

Instruction	Purpose
<code>RUN</code>	Execute a command while building the image
<code>ENV</code>	Define environment variables
<code>EXPOSE</code>	Document the port used by the application
<code>ENTRYPOINT</code>	Main command executed when the container starts
<code>CMD</code>	Default command or arguments

Understanding EXPOSE

Example:

```
EXPOSE 8080
```

This tells anyone reading the image:

This application normally listens on port **8080**.

Important

`EXPOSE` **does not publish the port.**

You still need:

```
docker run -p 8080:8080 product-service
```

to access the application from your computer.

Common Beginner Mistakes

Mistake 1

Building from the wrong folder.

Always run:

```
docker build .
```

inside the project directory that contains the Dockerfile.

Mistake 2

Forgetting `.dockerignore`.

This sends unnecessary files to Docker and slows down the build.

Mistake 3

Using random image names.

Instead of:

```
myimage
```

Use meaningful names:

```
product-service
```

```
order-service
```

```
inventory-service
```

Mistake 4

Confusing `EXPOSE` with `-p`.

Remember:

- `EXPOSE` documents the port inside the image.
- `-p` maps the container port to your computer.

Hands-on Lab

Exercise 1

Create a Dockerfile for your Spring Boot application.

Use:

```
FROM eclipse-temurin:21-jre

COPY app.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Exercise 2

Create a `.dockerignore` file.

```
target/

.idea/

.git/

*.log
```

Exercise 3

Build your image.

```
docker build -t product-service .
```

Exercise 4

Verify the image.

```
docker images
```

Exercise 5

Run the application.

```
docker run --name product-service -p 8080:8080 product-service
```

Open:

```
http://localhost:8080
```

Verify that the application starts successfully.

Key Takeaways

- ✓ A **Dockerfile** is a recipe used to build Docker images.
- ✓ `docker build` creates an image from a Dockerfile.
- ✓ `FROM` specifies the base image.
- ✓ `COPY` copies files into the image.
- ✓ `ENTRYPOINT` defines the startup command.
- ✓ `.dockerignore` excludes unnecessary files from the build context.

- ✓ Docker images are built in **layers**, allowing Docker to reuse cached layers and speed up future builds.
- ✓ The order of Dockerfile instructions affects build performance.

Self-Check Questions

1. What is a Dockerfile?
2. What does the `FROM` instruction do?
3. What is the purpose of the `COPY` instruction?
4. What does the `ENTRYPOINT` instruction define?
5. What is the Build Context in `docker build .`?
6. Why should every project include a `.dockerignore` file?
7. What is a Docker image layer?
8. Why does the order of Dockerfile instructions matter?

Summary

In this part, you learned how to build your own Docker images using a Dockerfile, how Docker uses image layers and caching to improve build performance, and why `.dockerignore` is an essential part of every project.

In **Part 5**, we will improve our Dockerfile using **Multi-Stage Builds**, dramatically reduce image size, introduce production-ready images, and prepare our Spring Boot microservices for deployment.

Part 5 — Multi-Stage Builds & Production Images

Course: Spring Boot & Spring Cloud — Professional Edition

Learning Objectives

By the end of this part, you will be able to:

- Understand the limitations of a single-stage Docker build.
 - Explain why build tools should not be included in production images.
 - Build a multi-stage Dockerfile for a Spring Boot application.
 - Compare single-stage and multi-stage images.
 - Understand the difference between build-time and runtime dependencies.
 - Create smaller, faster, and more secure production images.
 - Follow Docker best practices for Spring Boot applications.
-

Introduction

In the previous part, we built our first Docker image using a simple Dockerfile.

```
FROM eclipse-temurin:21-jre

COPY app.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

This works perfectly if you already have a compiled `app.jar`.

But in a real project, Docker usually starts with the **source code**, not the JAR file.

That means Docker must:

1. Download Maven dependencies.
2. Compile the project.
3. Run tests (optional).
4. Package the application.

5. Run the application.

How should we do this?

Many beginners create a **single-stage Dockerfile**.

Single-Stage Build

A typical beginner Dockerfile looks like this:

```
FROM maven:3.9-eclipse-temurin-21

WORKDIR /app

COPY . .

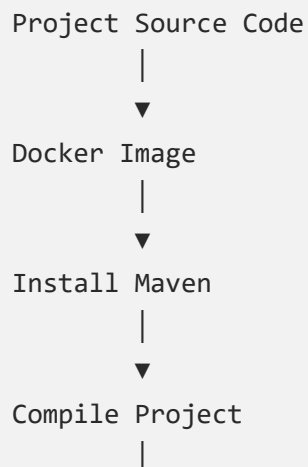
RUN mvn clean package

ENTRYPOINT ["java", "-jar", "target/product-service-0.0.1-SNAPSHOT.jar"]
```

It works.

So what's the problem?

What Happens During the Build?





Everything happens inside one image.

The Problem

The final image contains:

- Java Runtime □
- Spring Boot Application □
- Maven □
- Maven Cache □
- Source Code □
- Build Tools □

Do we need Maven to **run** the application?

No.

We only need Maven to **build** the application.

Build-Time vs Runtime

This is one of the most important concepts in Docker.

Build-Time

Used only while building the application.

Examples:

- Maven
- Gradle
- Source Code
- Unit Tests

- Build Plugins

Runtime

Needed after deployment.

Examples:

- Java Runtime
- Spring Boot JAR
- Configuration
- Environment Variables

Visual Comparison

Build-Time

Maven

Gradle

Source Code

Tests

↓

Runtime

Java Runtime

Application JAR

Configuration

Remember

Build-time tools should **never** be included in production images.

Why Is This a Problem?

A single-stage image is:

- Larger
- Slower to download
- Slower to deploy
- Less secure

Imagine deploying your application to 100 servers.

A larger image means:

- More network traffic
- More storage
- Longer deployments

Approximate Image Sizes

Build Type	Approximate Size
Single-stage	600–700 MB
Multi-stage	180–300 MB

The exact size depends on your project and the base images, but multi-stage builds are **significantly smaller**.

The Solution: Multi-Stage Builds

Docker allows us to use **multiple FROM statements**.

Each **FROM** creates a new build stage.

Stage 1:

- Build the application.

Stage 2:

- Copy only the JAR file.
- Ignore everything else.

Multi-Stage Workflow

Stage 1

Maven

Source Code

Compile

Package

|



Generated JAR

|



Stage 2

Java Runtime

+

Application JAR

|



Production Image

Notice that Maven and the source code never reach the final image.

Multi-Stage Dockerfile

Let's build our Spring Boot application.

```
# ----- Stage 1 : Build -----  
FROM maven:3.9-eclipse-temurin-21 AS builder  
  
WORKDIR /app
```

```
COPY pom.xml .

COPY src ./src

RUN mvn clean package -DskipTests

# ----- Stage 2 : Runtime -----
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

This is the Dockerfile we will use throughout the course.

Understanding Stage 1

```
FROM maven:3.9-eclipse-temurin-21 AS builder
```

We start with a Maven image.

It already contains:

- Java 21
- Maven

The stage is named:

```
builder
```

Later, we will copy files from this stage.

WORKDIR

```
WORKDIR /app
```

Creates (or switches to) the working directory.

Equivalent to:

```
cd /app
```

All following commands execute inside `/app`.

COPY

```
COPY pom.xml .
```

```
COPY src ./src
```

These commands copy:

- `pom.xml`
- `src`

into the Docker image.

RUN

```
RUN mvn clean package -DskipTests
```

This command executes **during image creation**.

It:

- Downloads dependencies.
- Compiles the code.
- Packages the JAR.

The generated JAR is stored in:

```
/app/target/
```

Question

Why do we use `-DskipTests`?

During development and CI/CD, tests should run before building the image.

When creating a production image, running the tests again often adds unnecessary build time because they have already passed in the pipeline.

In a real CI/CD pipeline, test execution and image creation are usually separate steps.

Understanding Stage 2

```
FROM eclipse-temurin:21-jre
```

This starts a brand-new image.

Notice:

There is **no Maven**.

There is **no source code**.

It contains only the Java Runtime Environment (JRE).

Copying from Another Stage

```
COPY --from=builder /app/target/*.jar app.jar
```

This is the key feature of multi-stage builds.

Docker copies **only** the generated JAR from the `builder` stage.

Everything else is discarded.

ENTRYPOINT

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

When the container starts, Docker executes:

```
java -jar app.jar
```

Exactly like running the application locally.

Comparing the Two Approaches

Single-Stage Build

```
Maven
Java
Source Code
Dependencies
Application
```

Everything remains in the final image.

Multi-Stage Build

```
Builder Stage
```

```
Maven
Source Code
Compile
Package
```

```
↓
```


Runtime Stage

Java Runtime
Application JAR

Much smaller.

Much cleaner.

Much safer.

Why Is It More Secure?

Every additional tool increases the attack surface.

If the image contains:

- Maven
- Git
- Shell tools
- Build utilities

an attacker has more tools available if the container is compromised.

A runtime-only image contains much less software.

Smaller images generally mean a smaller attack surface.

Best Practices for Spring Boot

Use a lightweight runtime image.

Good choices include:

- Eclipse Temurin JRE
- Amazon Corretto
- Eclipse Temurin with Alpine (if compatible)

Avoid installing unnecessary software inside the image.

Don't Copy Everything

Bad:

```
COPY . .
```

This copies:

- Source code
- IDE files
- Logs
- Git metadata
- Target folder

Better:

```
COPY pom.xml .
```

```
COPY src ./src
```

Copy only what you need.

Optimize Layer Caching

Instead of:

```
COPY . .
```

```
RUN mvn clean package
```

Use:

```
COPY pom.xml .
```

```
RUN mvn dependency:go-offline
```

```
COPY src ./src
```

```
RUN mvn clean package -DskipTests
```

Now Docker caches the downloaded dependencies.

If only the Java code changes, Docker reuses the cached dependency layer, making builds much faster.

Common Beginner Mistakes

Mistake 1

Using a Maven image in production.

This makes the image unnecessarily large.

Mistake 2

Copying the entire project with:

```
COPY . .
```

This increases the build context and invalidates Docker's cache more often.

Mistake 3

Keeping source code inside the production image.

Production images should contain only what is required to run the application.

Mistake 4

Using a JDK when only a JRE is needed.

A JDK includes compiler and development tools.

A JRE contains only what is required to run Java applications, making it smaller.

Hands-on Lab

Exercise 1

Create the following multi-stage Dockerfile:

```
FROM maven:3.9-eclipse-temurin-21 AS builder

WORKDIR /app

COPY pom.xml .

COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Exercise 2

Build the image.

```
docker build -t product-service .
```

Exercise 3

Verify the image.

```
docker images
```

Compare its size with a single-stage image if you created one.

Exercise 4

Run the application.

```
docker run -p 8080:8080 product-service
```

Open:

```
http://localhost:8080
```

Verify the application starts successfully.

Key Takeaways

- ✓ Multi-stage builds separate **building** from **running**.
- ✓ Build tools such as Maven should not be included in production images.
- ✓ Runtime images should contain only the Java runtime and the application JAR.
- ✓ Smaller images are faster to download, faster to deploy, and generally more secure.
- ✓ Layer caching can significantly reduce build times when Dockerfiles are organized correctly.

Self-Check Questions

1. What is the main problem with a single-stage Docker build?
2. What is the difference between build-time and runtime dependencies?
3. Why are multi-stage images usually smaller?
4. What does `COPY --from=builder` do?
5. Why is including Maven in a production image considered a bad practice?
6. Why is copying only `pom.xml` before `src` beneficial?
7. Why do many Docker builds use `-DskipTests`?
8. What are the main advantages of multi-stage builds?

Summary

You now understand why professional Docker images use **multi-stage builds** and how they separate the build environment from the runtime environment. This approach produces smaller, faster, and more secure images that are well suited for Spring Boot microservices.

In **Part 6**, we will introduce **Docker Compose**, learn how to run multiple microservices together, configure networks and environment variables, use health checks, and start the entire microservices platform with a single command.

Part 6 — Docker Compose: Running Multiple Containers Together

Course: Spring Boot & Spring Cloud — Professional Edition

Learning Objectives

By the end of this part, you will be able to:

- Understand why Docker Compose is needed.
 - Explain what a `docker-compose.yml` file is.
 - Understand the main sections of a Compose file.
 - Start and stop multiple containers with a single command.
 - Understand Docker Compose networking.
 - Configure environment variables.
 - Use named containers and service names.
 - Use the most common Docker Compose commands.
-

Introduction

So far, we have learned how to run **one container**.

Example:

```
docker run nginx

docker run postgres

docker run redis
```

This is fine for simple applications.

But what about a **Microservices Platform**?

Imagine Our Project

Our project contains:

Gateway

Product Service

Order Service

Inventory Service

Config Server

Eureka Server

Redis

PostgreSQL

Kafka

Zipkin

Would we start each service manually?

`docker run ...`

`docker run ...`

`docker run ...`

`docker run ...`

`docker run ...`

`docker run ...`

`docker run ...`


```
docker run ...
```

That would be slow, repetitive, and difficult to manage.

There must be a better solution.

The Solution

The solution is:

Docker Compose

Docker Compose allows us to describe the entire platform in a single file.

Instead of typing many commands, we write one configuration file.

Then execute one command.

What is Docker Compose?

Docker Compose is a tool for defining and running **multiple Docker containers**.

Instead of this:

Run PostgreSQL

↓

Run Redis

↓

Run Config Server

↓

Run Eureka

↓

Run Product Service

↓

Run Gateway

We simply execute:

```
docker compose up
```

Everything starts automatically.

What is docker-compose.yml?

Docker Compose reads a file named:

```
docker-compose.yml
```

This file describes:

- Services
- Networks
- Volumes
- Environment Variables
- Ports
- Startup Order

Think of it as the blueprint for your entire application.

Docker Compose Workflow

```
docker-compose.yml
```

|
▼

```
docker compose up
|
▼
Docker Engine
|
▼
Creates Containers
|
▼
Starts Entire Platform
```

Our First Compose File

Let's create a simple example.

```
services:

  nginx:
    image: nginx
    ports:
      - "8080:80"

  redis:
    image: redis

  postgres:
    image: postgres
```

Save it as:

```
docker-compose.yml
```

Understanding the File

Every service appears under:

```
services:
```

Each service has:

- Name
- Image
- Configuration

Example:

```
services:

  nginx:
    image: nginx
```

The service name is:

```
nginx
```

The image is:

```
nginx
```

Starting the Platform

Run:

```
docker compose up
```

Docker Compose will:

- Download missing images.
- Create containers.
- Create a network.
- Start all containers.

All with one command.

Running in Background

Normally, Docker Compose displays logs.

To run it in the background:

```
docker compose up -d
```

The **-d** option means:

Detached Mode

Exactly like `docker run -d`.

Viewing Running Containers

```
docker compose ps
```

Example:

```
NAME
```

```
STATUS
```

```
PORTS
```

Notice this is different from:

```
docker ps
```

Compose only displays containers belonging to the current project.

Viewing Logs

Show logs from all services:

```
docker compose logs
```

Follow live logs:

```
docker compose logs -f
```

Stopping the Platform

Stop all containers:

```
docker compose stop
```

Containers still exist.

They are only stopped.

Starting Again

```
docker compose start
```

The containers continue from where they stopped.

Restarting

```
docker compose restart
```

Restarts every service.

Removing Everything

When you are finished:

```
docker compose down
```

Docker Compose:

- Stops containers
- Removes containers
- Removes the project network

Your images remain available.

Compose Commands Summary

Command	Description
<code>docker compose up</code>	Create and start all services
<code>docker compose up -d</code>	Start in background
<code>docker compose ps</code>	Show project containers
<code>docker compose logs</code>	View logs
<code>docker compose logs -f</code>	Follow logs
<code>docker compose stop</code>	Stop services
<code>docker compose start</code>	Start stopped services
<code>docker compose restart</code>	Restart services
<code>docker compose down</code>	Stop and remove services

Service Names

Consider this file:

```
services:
```

```
postgres:
  image: postgres

redis:
  image: redis
```

Docker Compose automatically creates:

postgres

redis

These names become important.

Containers can communicate using these names.

For example:

Instead of writing:

localhost

The Product Service can connect to PostgreSQL using:

postgres

Similarly,

Redis can be reached using:

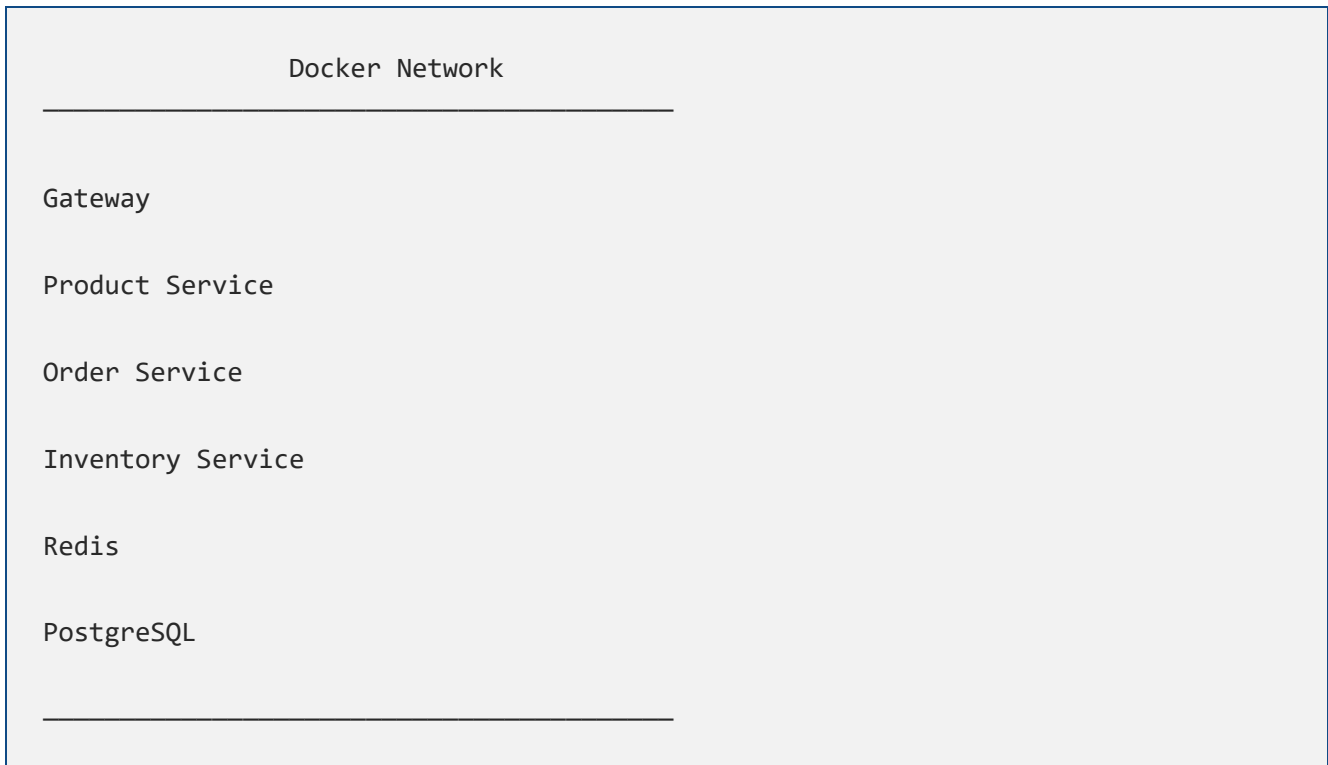
redis

No IP addresses are required.

Docker Compose Networking

One of the best features of Docker Compose is automatic networking.

When Compose starts, it creates a private network.



Every service joins the same network automatically.

Why Is This Important?

Suppose Product Service needs PostgreSQL.

Without Docker Compose:

You must know the PostgreSQL container IP address.

With Docker Compose:

Simply use:

```
postgres
```

Docker automatically resolves the name.

Example

Spring Boot configuration:

```
spring.datasource.url=jdbc:postgresql://postgres:5432/productdb
```

Notice:

Not:

```
localhost
```

Not:

```
192.168.1.25
```

Instead:

```
postgres
```

Because Docker Compose provides automatic DNS.

Environment Variables

Instead of hardcoding configuration, Compose can provide values.

Example:

```
environment:  
  SPRING_PROFILES_ACTIVE: docker
```

Spring Boot automatically receives:

```
SPRING_PROFILES_ACTIVE=docker
```

Another example:

```
environment:
```

```
  DB_HOST: postgres
```

```
  DB_PORT: 5432
```

The application can read these variables.

Port Mapping

Exactly like Docker.

Example:

```
ports:
```

```
  - "8080:8080"
```

Meaning:

Computer Port

8080

↓

Container Port

8080

Naming Containers

Compose automatically generates names.

Example:

```
microservices-postgres
```

```
microservices-redis
```

```
microservices-gateway
```

These names are based on:

- Project name
- Service name

Usually, you do not need to specify `container_name`.

Best Practice

Avoid using `container_name` unless there is a specific requirement.

Let Docker Compose generate names automatically. This makes scaling and project isolation easier.

Common Beginner Mistakes

Mistake 1

Running individual containers manually.

If a project uses Docker Compose:

Use:

```
docker compose up
```

Not multiple `docker run` commands.

Mistake 2

Using localhost between containers.

Inside Docker Compose:

Use the service name.

Example:

```
postgres
```

instead of

```
localhost
```

Mistake 3

Forgetting `docker compose down`.

Stopped containers and unused networks accumulate over time.

Clean them when you finish.

Mistake 4

Editing containers manually.

Containers should be recreated from the Compose file.

Treat the Compose file as the source of truth.

Hands-on Lab

Create the following file.

```
services:

  nginx:
    image: nginx
    ports:
```

```
- "8080:80"
```

```
redis:
```

```
image: redis
```

```
postgres:
```

```
image: postgres
```

Run:

```
docker compose up
```

Observe:

- Images are downloaded.
- Network is created.
- Three containers start.

Open:

```
http://localhost:8080
```

Verify that the NGINX welcome page appears.

Display the running services:

```
docker compose ps
```

View the logs:

```
docker compose logs
```

Stop everything:

Key Takeaways

- ✓ Docker Compose manages multiple containers using a single configuration file.
- ✓ `docker-compose.yml` describes your application's services.
- ✓ One command can start the entire platform.
- ✓ Docker Compose automatically creates a shared network.
- ✓ Containers communicate using **service names**, not IP addresses.
- ✓ Environment variables should be provided through the Compose file instead of being hardcoded.
- ✓ `docker compose down` cleans up the project when you are finished.

Self-Check Questions

1. Why do we use Docker Compose instead of multiple `docker run` commands?
2. What is the purpose of the `docker-compose.yml` file?
3. What does `docker compose up` do?
4. What is the difference between `docker compose up` and `docker compose up -d`?
5. Why should containers use service names instead of IP addresses?
6. What command displays the containers managed by the current Compose project?
7. What does `docker compose down` remove?
8. Why are environment variables preferred over hardcoded configuration values?

Summary

You have learned the fundamentals of Docker Compose and how it simplifies running multiple containers together. You now understand how Compose defines services, creates a shared network,

provides automatic service discovery, and manages the lifecycle of an entire application with just a few commands.

In **Part 7**, we will build a **production-ready Docker Compose file** for our Spring Boot microservices platform. We'll connect the Config Server, Eureka Server, API Gateway, Product Service, PostgreSQL, and Redis, introduce **health checks**, use **depends_on**, inject **Spring Cloud Config** and **Eureka** URLs through environment variables, and discuss important production best practices. This final part directly addresses the professional deployment objectives of this session.

Part 7 — Docker Compose for Spring Boot Microservices (Production-Ready)

Course: Spring Boot & Spring Cloud — Professional Edition

Learning Objectives

By the end of this part, you will be able to:

- Containerize an entire Spring Boot microservices platform.
 - Configure Spring Cloud Config Server, Eureka Server, API Gateway, and Product Service in Docker Compose.
 - Use environment variables instead of hardcoded URLs.
 - Implement Docker Health Checks using Spring Boot Actuator.
 - Understand `depends_on` and `service_healthy`.
 - Understand what Docker Compose guarantees—and what it does **not** guarantee.
 - Apply production best practices for Docker Compose.
-

Introduction

In the previous part, we learned how Docker Compose starts multiple containers.

Now let's apply that knowledge to our **Microservices Platform**.

Instead of running:

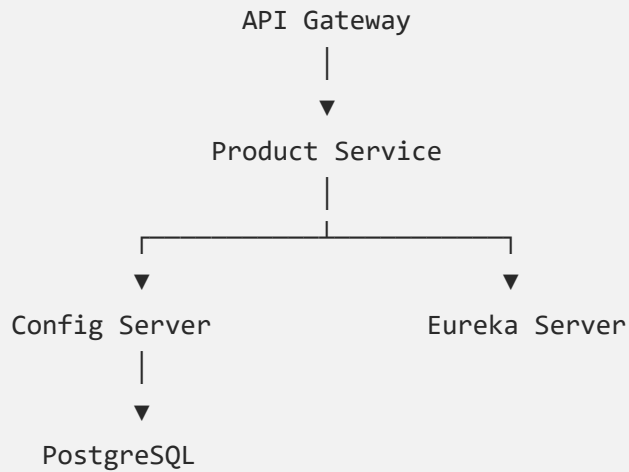
- Config Server
- Eureka Server
- API Gateway
- Product Service
- PostgreSQL
- Redis

one by one...

Docker Compose will start everything automatically.

Our Platform

We will use the following services.



Note

For simplicity, this tutorial includes one business service (**Product Service**). Adding **Order Service**, **Inventory Service**, or **Notification Service** follows exactly the same pattern.

Step 1 — Build Docker Images

Before using Docker Compose, each service must have its own Docker image.

For example:

```
config-server
```

```
↓
```

```
Docker Image
```

```
-----
```

```
eureka-server
```

↓

Docker Image

gateway

↓

Docker Image

product-service

↓

Docker Image

Example:

```
docker build -t config-server .
```

```
docker build -t eureka-server .
```

```
docker build -t gateway .
```

```
docker build -t product-service .
```

Later, these images can also be stored in Docker Hub or another container registry.

Step 2 — Basic Compose File

Let's begin with the structure.

```
services:

  config-server:

  eureka-server:

  gateway:

  product-service:

  postgres:

  redis:
```

Each service will have its own configuration.

Step 3 — Config Server

```
config-server:
  image: config-server
  ports:
    - "8888:8888"
```

Nothing special yet.

Step 4 — Eureka Server

```
eureka-server:
  image: eureka-server
  ports:
    - "8761:8761"
```

Step 5 — API Gateway

```
gateway:
  image: gateway
  ports:
    - "8080:8080"
```

Step 6 — Product Service

```
product-service:
  image: product-service
```

At this point, the service starts...

But it doesn't know:

- Where Config Server is.
- Where Eureka Server is.
- Where PostgreSQL is.

Let's fix that.

Why Should We Avoid Hardcoding?

Many beginners write:

```
spring.config.import=configserver:http://localhost:8888

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

This works only on your computer.

Inside Docker,

localhost means the current container, not another container.

So this configuration is incorrect.

The Correct Solution

Use Docker service names.

Config Server

↓

config-server

Eureka Server

↓

eureka-server

PostgreSQL

↓

postgres

Docker automatically resolves these names.

Environment Variables

Instead of hardcoding values,

Docker Compose injects them.

Example:

```
environment:  
  CONFIG_SERVER_URL: http://config-server:8888
```

Inside Spring Boot:

```
${CONFIG_SERVER_URL}
```

The value comes from Docker Compose.

Another example:

```
environment:  
  EUREKA_SERVER: http://eureka-server:8761/eureka
```

Spring Boot reads:

```
${EUREKA_SERVER}
```

No hardcoded URLs.

Passing Database Information

```
environment:  
  
  DB_HOST: postgres  
  
  DB_PORT: 5432  
  
  DB_NAME: productdb  
  
  DB_USERNAME: postgres
```

```
DB_PASSWORD: postgres
```

Spring Boot reads these values when starting.

Spring Boot Configuration

Example:

```
spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}

spring.datasource.username=${DB_USERNAME}

spring.datasource.password=${DB_PASSWORD}
```

Notice:

No IP addresses.

No localhost.

Everything comes from environment variables.

Why Environment Variables?

Advantages:

- Easy to change
- No code changes
- Same application works in:
 - Development
 - Testing
 - Production

Only the environment variables change.

Health Checks

Imagine Docker starts Product Service immediately.

Product Service tries to connect to Config Server.

But Config Server is still starting.

Result:

```
Connection Refused
```

The application fails.

How can Docker know whether a service is ready?

The answer:

Health Checks

Spring Boot Actuator

Earlier in this course we added:

```
spring-boot-starter-actuator
```

One endpoint is:

```
/actuator/health
```

Example:

```
http://localhost:8888/actuator/health
```

Response:

```
{  
  "status": "UP"
```

```
}
```

Docker can use this endpoint.

Docker Health Check

Example:

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8888/actuator/health"]
  interval: 10s
  timeout: 5s
  retries: 5
```

Docker repeatedly checks:

Is the application healthy?

If the endpoint returns:

UP

Docker marks the container as:

Healthy

Understanding Health Check Options

```
interval: 10s
```

Check every 10 seconds.

```
timeout: 5s
```

Wait at most 5 seconds for a response.

```
retries: 5
```

After five failures,

Docker marks the container as unhealthy.

depends_on

Suppose Gateway needs Eureka.

Without coordination,

Gateway may start first.

Result:

```
Cannot connect to Eureka
```

Docker Compose provides:

```
depends_on:
```

Example:

```
gateway:
  depends_on:
    eureka-server:
      condition: service_healthy
```

Meaning:

Start Gateway **only after** Eureka becomes healthy.

Another Example

Product Service depends on:

- Config Server
- Eureka
- PostgreSQL

```
depends_on:

  config-server:
    condition: service_healthy

  eureka-server:
    condition: service_healthy

  postgres:
    condition: service_healthy
```

Docker waits until all are healthy.

Important Limitation of depends_on

This is one of the most misunderstood Docker Compose features.

Many developers believe:

```
depends_on

↓

Everything is guaranteed forever.
```

This is **NOT true**.

Suppose:

Config Server

↓

Healthy

↓

Gateway Starts

↓

Config Server Crashes

Will Docker restart Gateway automatically?

No.

`depends_on` only affects **startup order**.

It does **not**:

- Monitor dependencies continuously.
- Restart dependent services.
- Recover failed applications.

Your application should be able to reconnect if another service becomes temporarily unavailable.

Remember

`depends_on` controls **startup order**, not runtime dependency management.

Complete Example

```
services:

  config-server:
    image: config-server
    ports:
      - "8888:8888"

  eureka-server:
    image: eureka-server
    ports:
      - "8761:8761"

  gateway:
    image: gateway
    ports:
      - "8080:8080"

  environment:
    CONFIG_SERVER_URL: http://config-server:8888
    EUREKA_SERVER: http://eureka-server:8761/eureka

  depends_on:
    config-server:
      condition: service_healthy
    eureka-server:
      condition: service_healthy
```

This example demonstrates the main concepts.

In a real project, additional configuration is usually required.

Development vs Production Compose

Development Compose often includes:

- Debug ports
- Volume mounts
- Local source code

- Hot reload

Example:

```
volumes:  
- ./src:/app/src
```

This allows live code changes without rebuilding the image.

Production Compose usually:

- Uses prebuilt images
 - No source code
 - No debug ports
 - No IDE files
 - Smaller images
 - Better security
-

Production Security

Containers should **not** run as the root user.

Instead:

```
RUN adduser --system spring  
  
USER spring
```

Running as a non-root user reduces the impact of a security breach.

Best Practice

Production containers should run with the minimum privileges required.

Best Practices

- ✓ Use Multi-Stage Builds.

- ✓ Use environment variables.
 - ✓ Never hardcode URLs.
 - ✓ Use Spring Boot Actuator.
 - ✓ Add health checks.
 - ✓ Use meaningful service names.
 - ✓ Keep images small.
 - ✓ Run containers as non-root users.
 - ✓ Use explicit image versions instead of `latest`.
 - ✓ Keep development and production Compose files separate.
-

Common Beginner Mistakes

Mistake 1

Using:

```
localhost
```

between containers.

Always use the service name.

Mistake 2

Hardcoding Config Server URLs.

Always use environment variables.

Mistake 3

Assuming `depends_on` guarantees runtime availability.

It only controls startup order.

Mistake 4

Running containers as root in production.

Use a dedicated application user whenever possible.

Mistake 5

Using the same Docker Compose file for development and production.

Development and production environments have different requirements.

Hands-on Lab

Build a Docker Compose file that includes:

- Config Server
- Eureka Server
- PostgreSQL
- Gateway
- Product Service

Requirements:

1. Use environment variables.
2. Add health checks.
3. Configure `depends_on`.
4. Use meaningful service names.
5. Verify that all services start successfully.

Run:

```
docker compose up -d
```

Verify:

```
docker compose ps
```

Open:

- `http://localhost:8761` (Eureka Dashboard)
- `http://localhost:8080` (Gateway)
- `http://localhost:8888` (Config Server)

Confirm that every container reports a **healthy** status.

Key Takeaways

- ✓ Docker Compose can start an entire microservices platform with a single command.
- ✓ Service names act as hostnames inside the Docker network.
- ✓ Environment variables make applications portable across environments.
- ✓ Health checks ensure services are ready before dependent services start.
- ✓ `depends_on` controls startup order but does not manage runtime failures.
- ✓ Production images should be small, secure, and run as non-root users.

Self-Check Questions

1. Why should service URLs not be hardcoded in Spring Boot applications?
2. Why does `localhost` not work for communication between Docker containers?
3. What is the purpose of Spring Boot Actuator in Docker Compose?
4. What does a Docker health check do?
5. What is the purpose of `depends_on`?

6. What does `condition: service_healthy` mean?
 7. What are the limitations of `depends_on`?
 8. Why should production containers avoid running as the root user?
-

Session 9 Summary

Congratulations! ☐

In this session, you learned how to package Spring Boot applications into Docker images and run an entire microservices platform using Docker Compose.

You can now:

- Explain the difference between Docker images and containers.
- Build Docker images using Dockerfiles.
- Optimize images with multi-stage builds.
- Use `.dockerignore` and Docker layer caching.
- Manage containers with Docker CLI.
- Create Docker Compose files.
- Configure networking and environment variables.
- Implement health checks with Spring Boot Actuator.
- Coordinate service startup with `depends_on`.
- Apply production-ready Docker best practices.

These skills provide the foundation for the next session, where you'll move beyond a single machine and learn how to orchestrate containers at scale using **Kubernetes**.